

Tömbök, listák és foreach

A mai óra technikai témái:

- egydimenziós tömbök,
- 0-alapú indexelés,
- tömbök létrehozása és bejárása,
- többdimenziós tömbök,
- Length és GetLength() ,
- List<T> ,
- Add() , Remove() , Count , Capacity ,
- foreach .

1. Tömbök

A tömb segítségével **azonos típusú több adatot** kezelhetünk egyetlen változón keresztül.

```
int[] numbers = new int[10];
```

Ez egy 10 elemű, egész számokat tároló tömb.

0-alapú indexelés

C#-ban az indexelés **0-tól indul**.

Elem:	[?]	[?]	[?]	[?]	[?]
Index:	0	1	2	3	4

Egy N elemű tömbnél:

```
első index = 0  
utolsó index =  $N - 1$ 
```

Például egy 10 elemű tömb érvényes indexei:

```
0 ... 9
```

Ez hibás:

```
numbers[10] = 42;
```

mert nincs `10` indexű elem.

2. Tömb deklarálása és létrehozása

Csak deklaráció

```
bool[] myBools;
```

Itt még csak egy tömbre hivatkozó változót deklaráltunk.

Tömb létrehozása

```
myBools = new bool[2];
```

Most létrejön a 2 elemű `bool` tömb.

Deklaráció és létrehozás egyszerre

```
int[] myNums = new int[10];
```

Létrehozás elemek felsorolásával

```
char[] someChars = { 'a', 'b', 'c' };
```

A tömb mérete ebben az esetben automatikusan 3.

3. Alapértelmezett értékek

Ha csak a méretet adjuk meg:

```
int[] values = new int[5];
```

a C# minden elemet az adott típus alapértelmezett értékével tölt fel.

Típus	Alapértelmezett érték
int	0
double	0.0
bool	false
char	'\0'
string	null
referencia típusok	null

Például:

```
bool[] values = new bool[3];  
  
Console.WriteLine(values[0]);
```

eredmény:

```
False
```

4. Tömb elemeinek írása és olvasása

Értékadás

```
myBools[1] = true;
```

Mivel az indexelés 0-tól indul, ez a **második elem**.

Lekérdezés

```
bool someBool = myBools[0];
```

Ez az első tömbelem értékét olvassa ki.

Példa

```
int[] numbers = new int[3];

numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;

Console.WriteLine(numbers[1]);
```

eredmény:

```
20
```

5. Tömb mérete — Length

A tömb elemszámát a `Length` adja meg.

```
int[] myNums = new int[10];

int count = myNums.Length;
```

Ekkor:

```
count = 10
```

Az utolsó érvényes index:

```
myNums.Length - 1
```

Ezért a tipikus tömbbejárás:

```
for (int i = 0; i < myNums.Length; i++)
{
    Console.WriteLine(myNums[i]);
}
```

Nem:

```
i <= myNums.Length
```

mert ekkor a végén nem létező indexhez próbálnánk hozzáférni.

6. Tömb feltöltése ciklussal

A tömbök használata szorosan kapcsolódik a ciklusokhoz.

```
int[] numbers = new int[10];

for (int i = 0; i < numbers.Length; i++)
{
    numbers[i] = i * i;
}
```

A tömb tartalma:

0, 1, 4, 9, 16, 25, 36, 49, 64, 81

A fontos minta:

```
array[i]
```

ahol `i` az aktuálisan feldolgozott elem indexe.

7. Többdimenziós tömbök

Kétdimenziós tömb segítségével táblázatos, mátrix-szerű adatokat tárolhatunk.

```
int[,] matrix =
{
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};
```

Ez így képzelhető el:

```
1 2 3
4 5 6
7 8 9
```

Egy elemhez két index szükséges:

```
matrix[sor, oszlop]
```

Példa:

```
Console.WriteLine(matrix[1, 2]);
```

eredmény:

6

8. N dimenzió → N index

Egydimenziós tömb:

```
int[] a;
```

elem:

```
a[i]
```

Kétdimenziós tömb:

```
int[,] matrix;
```

elem:

```
matrix[i, j]
```

Háromdimenziós tömb:

```
string[, ,] mysteriousArray =  
    new string[51, 33, 23];
```

elem:

```
mysteriousArray[9, 11, 13] = "The Truth is out there";
```

Általánosan:

N dimenziós tömb → N darab index.

9. Length és GetLength()

```
int[,] matrix = new int[11, 13];
```

A tömb:

11 sor × 13 oszlop

Teljes elemszám

```
int count = matrix.Length;
```

eredmény:

143

Egy dimenzió hossza

```
int numRows = matrix.GetLength(0);  
int numCols = matrix.GetLength(1);
```

eredmény:

```
numRows = 11  
numCols = 13
```

A dimenziók indexelése is 0-tól indul:

```
GetLength(0) → első dimenzió  
GetLength(1) → második dimenzió
```

10. Kétdimenziós tömb bejárása

Két dimenzióhoz általában két egymásba ágyazott ciklust használunk.

```
int[,] matrix = new int[11, 13];

int numRows = matrix.GetLength(0);
int numCols = matrix.GetLength(1);

for (int i = 0; i < numRows; i++)
{
    for (int j = 0; j < numCols; j++)
    {
        matrix[i, j] = i + j;
    }
}
```

A külső ciklus:

```
i
```

a sorokat járja be.

A belső ciklus:

```
j
```

az oszlopokat járja be.

Az aktuális elem:

```
matrix[i, j]
```

11. Mátrix kiírása

```
for (int i = 0; i < matrix.GetLength(0); i++)
{
    for (int j = 0; j < matrix.GetLength(1); j++)
    {
        Console.Write(matrix[i, j] + " ");
    }

    Console.WriteLine();
}
```

A belső ciklus kiír egy sort, majd:

```
Console.WriteLine();
```

új sort kezd.

12. Listák — List<T>

A tömb mérete létrehozáskor rögzített.

```
int[] numbers = new int[10];
```

A List<T> ezzel szemben **dinamikus gyűjtemény**: a benne lévő elemek száma futás közben változhat.

Üres lista

```
List<int> listOfNumbers = new List<int>();
```

Kezdőelemekkel

```
List<string> listOfStrings =  
    new List<string> { "Trust", "No", "One" };
```

A generikus típus:

```
<int>  
<string>  
<char>
```

megadja, milyen típusú elemeket tárol a lista.

13. Lista műveletek

Add ()

Új elem hozzáadása:

```
List<char> letters = new List<char>();  
  
letters.Add('a');  
letters.Add('b');
```

Index szerinti módosítás

```
letters[1] = 'c';
```

A lista tartalma:

```
a, c
```

A lista tehát ugyanúgy indexelhető, mint a tömb.

14. Remove()

```
letters.Remove('a');
```

A `Remove()` a megadott érték **első előfordulását** törli.

Példa:

```
List<int> values =  
    new List<int> { 1, 2, 2, 3 };  
  
values.Remove(2);
```

Eredmény:

```
1, 2, 3
```

Csak az első `2` tűnik el.

15. Count és Capacity

Count

Az aktuálisan tárolt elemek száma.

```
int count = letters.Count;
```

Capacity

A lista belső tárolójának aktuális kapacitása.

```
int capacity = letters.Capacity;
```

Például lehet:

```
Count = 3  
Capacity = 4
```

Ez azt jelenti, hogy:

- 3 tényleges elem van a listában,
- jelenleg 4 elem számára van lefoglalt hely.

A legtöbb feladatban a `Count` az érdekesebb érték.

16. Lista bejárása for ciklussal

```
List<char> letters = new List<char>();  
  
letters.Add('a');  
letters.Add('b');  
letters[1] = 'c';  
  
for (int i = 0; i < letters.Count; i++)  
{  
    Console.Write(letters[i]);  
}
```

eredmény:

```
ac
```

Fontos:

```
tömb → Length  
lista → Count
```

17. Tömb és lista összehasonlítása

Tulajdonság	Tömb	List<T>
Méret	fix	dinamikus
0-alapú index	igen	igen
Index szerinti elérés	igen	igen
Elem hozzáadása	nincs Add()	Add()
Elem törlése	közvetlenül nincs	Remove()
Elemszám	Length	Count
for használható	igen	igen
foreach használható	igen	igen

Ha előre ismert az elemszám, a tömb jó választás lehet.

Ha az elemszám futás közben változik, a lista általában kényelmesebb.

18. foreach

A `foreach` ciklus gyűjtemények bejárására használható.

```
int[] numbers = { 10, 20, 30 };

foreach (int n in numbers)
{
    Console.WriteLine(n);
}
```

eredmény:

```
10
20
30
```

A `foreach` automatikusan kezeli a léptetést.

Nincs szükség:

```
i++
```

utasításra, és nem kell külön indexet kezelnünk.

19. Mit kap a foreach változó?

```
foreach (int n in numbers)
{
    Console.WriteLine(n);
}
```

A `n` nem az index.

A `n` közvetlenül az aktuális elem értékét kapja.

Ha:

```
int[] numbers = { 5, 10, 15 };
```

akkor `n` értékei:

```
5
10
15
```

20. Példa — négyzetek listába gyűjtése

```
int[] numbers =
    new int[] { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 };

List<int> squares = new List<int>();

foreach (int n in numbers)
{
    int squareNumber = n * n;

    Console.WriteLine($"{n}^2 = {squareNumber}");

    squares.Add(squareNumber);
}
```

A program:

1. végigjárja a `numbers` tömböt,
2. kiszámítja minden elem négyzetét,
3. kiírja az eredményt,
4. hozzáadja a négyzetet a `squares` listához.

21. for vagy foreach ?

foreach

Akkor kényelmes, ha:

- minden elemet egyszer szeretnénk feldolgozni,
- nincs szükségünk az indexre.

```
foreach (string name in names)
{
    Console.WriteLine(name);
}
```

for

Jobb lehet, ha:

- szükségünk van az indexre,
- index alapján akarunk módosítani,
- csak bizonyos elemeket akarunk feldolgozni,
- visszafelé akarunk haladni.

```
for (int i = 0; i < numbers.Length; i++)
{
    numbers[i] = numbers[i] * 2;
}
```

22. foreach és módosítás

A jegyzetben használt egyszerű értéktípusos példánál a `foreach` ciklusváltozó az aktuális elem értékét kapja.

Ezért ezt nem használjuk az eredeti tömbelem módosítására:

```
foreach (int n in numbers)
{
    // n = n * 2; // nem az eredeti tömbelem index szerinti módosítása
}
```

Ha magukat a tömbelemeket akarjuk módosítani, használjunk indexet:

```
for (int i = 0; i < numbers.Length; i++)
{
```

```
    numbers[i] *= 2;  
}
```

23. Tipikus hibák

Hibás index

```
int[] numbers = new int[5];  
  
numbers[5] = 10;
```

Az érvényes indexek:

0, 1, 2, 3, 4

<= használata **<** helyett

Hibás:

```
for (int i = 0; i <= numbers.Length; i++)
```

Helyes:

```
for (int i = 0; i < numbers.Length; i++)
```

Length és Count keverése

Tömb:

```
numbers.Length
```

Lista:

```
numbers.Count
```

Length félreértése mátrixnál

```
matrix.Length
```

a teljes elemszám.

Dimenziók:

```
matrix.GetLength(0)  
matrix.GetLength(1)
```

24. Feladatokhoz szükséges minták

Kártyapakli létrehozása

A kártyalapok előállításánál két gyűjteményt kombinálunk:

4 szín × 13 érték = 52 lap

Ehhez természetes megoldás két egymásba ágyazott ciklus.

```
for (...)
{
    for (...)
    {
        // lap előállítása
    }
}
```

25. Fisher–Yates keverés

A feladatban egy tömb elemeit kell véletlenszerűen összekeverni.

A kulcsművelet két elem felcserélése:

```
string temp = cards[i];  
cards[i] = cards[j];  
cards[j] = temp;
```

A j indexet megfelelő tartományból véletlenszerűen választjuk.

A fontos gondolat:

A tömb elemeit nem újra létrehozzuk, hanem indexek segítségével felcseréljük.

26. Keresés tömbben vagy listában

Egyszerű lineáris keresés:

```
for (int i = 0; i < words.Length; i++)
{
    if (words[i] == searchedWord)
    {
        // megtaláltuk
    }
}
```

Ha az első előfordulás indexére is szükségünk van:

```
bool found = false;
int foundIndex = -1;
```

A -1 azért praktikus, mert nem lehet érvényes tömbindex.

27. Mátrix transzponálása

Eredeti:

```
1 2 3
4 5 6
7 8 9
```

Transzponált:

```
1 4 7
2 5 8
3 6 9
```

Az indexelési szabály:

$\text{eredeti}[i, j] \rightarrow \text{transzponált}[j, i]$

Kódban:

```
transposed[j, i] = matrix[i, j];
```

Ez a feladat közvetlenül gyakoroltatja:

- a kétdimenziós indexelést,
- a `GetLength()` használatát,
- az egymásba ágyazott ciklusokat.

28. Gyors összefoglaló

Tömb

```
int[] numbers = new int[10];  
numbers[0] = 42;  
int n = numbers.Length;
```

Kétdimenziós tömb

```
int[,] matrix = new int[5, 10];  
  
matrix[2, 3] = 42;  
  
int rows = matrix.GetLength(0);  
int cols = matrix.GetLength(1);
```

Lista

```
List<int> numbers = new List<int>();  
  
numbers.Add(42);  
numbers.Remove(42);  
  
int n = numbers.Count;
```

foreach

```
foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

29. Amit tudni kell a feladatok előtt

- A tömbök indexelése 0-tól indul.
- Egy N elemű tömb utolsó indexe $N - 1$.
- Tömb elemszáma: `Length`.
- Többdimenziós tömbnél N dimenzióhoz N index tartozik.
- Egy adott dimenzió hossza: `GetLength(index)`.
- Mátrixokhoz általában egymásba ágyazott ciklus kell.
- A lista dinamikusan növelhető és csökkenthető.
- Lista aktuális elemszáma: `Count`.
- `Add()` új elemet ad hozzá.
- `Remove()` az első megfelelő elemet törli.
- `foreach` közvetlenül az elemeket járja be.
- Ha indexre vagy index szerinti módosításra van szükség, a `for` gyakran jobb választás.

A fő kérdés minden feladatnál: milyen adatstruktúrában tároljuk az adatokat, és hogyan járjuk be őket?